Emil GROUP

Coding Challenge

Sr Software Engineer

Introduction

This assignment is aimed at experienced engineers who want to showcase full-stack architecture, clean-code craftsmanship, and a healthy dose of product thinking.

You will design one cohesive solution containing:

1. A modern Claim Service that stores and manages insurance claims.
2. An embeddable claim-filing funnel through which existing customers can submit a new claim.
3. A drag-&-drop designer that lets non-developers change the funnel without triggering a code deploy.

Treat the exercise like a mini green-field project at EMIL: cloud-native, well-documented, and maintainable.

Feel free to use productivity tools such as Cursor or any other AI-assisted IDE during development—just note any autogenerated code or prompts in your README.

Time required

You can spend as much time as you want to. If this challenge contains new things for you, we recon a few evenings should be sufficient for completing it.

Submission

Send us the repo link (public or private with access) as soon as possible. We'll invite you to a 60-90-minute tech-deep-dive to walk through decisions, code, and trade-offs.

Challenge

1. Claim Service (Backend)

- **Stack**: NestJS v10+ in TypeScript (preferred) or Go.
- **Responsibilities**
  - CRUD
  - Lifecycle (open → in-review → closed) for Claim entities.
  - Expose both REST and gRPC APIs

2. Embeddable Claim-Filing Funnel (Frontend Widget)

- **Stack**: React 18 + Vite or Next.js (client-side only is fine), Tailwind.
- **Goal**: A 3-to-5-step wizard that collects enough data and creates an immediate Claim for an existing user. Assume the calling site provides a valid userId or JWT — no need to build policy purchase or user onboarding.
- **Embedding**: Deliver as a single JS bundle that site owners can `<script>` include. Provide a minimal HTML demo page showing integration.

3. Funnel Designer (Low-Code CMS)

- **Goal**: Non-devs can change copy, fields, and order of steps without redeploying the widget.
- **Minimal viable spec**
  - Store configuration as JSON (your DB, object store, or even a local file).
- **Stretch goals**: Drag-and-drop editor, versioning, A/B toggles.

For each part of the assignment, consider which additional elements or refinements will elevate your solution to a production-ready, professional standard.

Deliverables

1. **Private code repository**
2. **README**
3. **Demo recording or screenshots** *optional but appreciated*.

Criteria for evaluation

Architecture
Code Quality
DevOps
Security
Product Thinking
Documentation
Creativity

We value *direction over completeness*. If you cut scope, document it.

Above all, submit a solution you're genuinely proud to showcase—this is your chance to demonstrate both engineering skill and craftsmanship.

- **Team EMIL**